RAG selettivo per benchmark su Qwen3-14B: mai peggio del modello base, guadagno piccolo e non significativo su ARC-it

Giovanni Del Papa — Ricercatore indipendente, Italia (progetto Lo Scienziato Dilettante GDP)

Preprint, ottobre 2026 — v1.1

Citazione: Del Papa, G. (2026). DOI: 10.5281/zenodo.23674921

Sommario

Valutiamo se un retrieval-augmented generation (RAG) leggero basato su BM25 su corpora Wikipedia migliori un modello linguistico aperto di medie dimensioni, Qwen3-14B (MLX 8-bit, modalità non-thinking, decoding greedy), eseguito in locale su un Mac mini. Anziché attivare il RAG ovunque, proponiamo un gate selettivo per benchmark: il RAG viene acceso per un benchmark solo se migliora un insieme di sviluppo disgiunto, e — per le domande a scelta multipla — solo per le singole domande in cui il punteggio di retrieval è alto e il modello senza RAG è poco sicuro. Il gate è scelto esclusivamente sul dev; il test è eseguito una sola volta. Su ARC italiano (n = 1132) l'accuratezza passa da 0,9046 a 0,9090 (+0,44 punti; 5 risposte corrette in più, 0 rotte; McNemar esatto p = 0,0625; bootstrap 95% [+0,09; +0,88] pt). Il RAG sempre attivo, al contrario, peggiora ARC-it a 0,8843 (−2,03 pt; 19 corrette in più, 42 rotte; p = 0,0044). Su GSM8K e IFEval il RAG resta spento perché non aiutava in modo affidabile sul dev, e la configurazione finale coincide con il modello base. Il risultato principale è quindi conservativo: il sistema selettivo non è mai peggiore del modello base, con un guadagno piccolo e non significativo al livello 0,05.

1. Introduzione

Il RAG è spesso presentato come un miglioramento "gratuito" dei modelli linguistici. In pratica, quando la conoscenza richiesta è già nel modello, i brani recuperati possono distrarlo e peggiorare le risposte. In un lavoro precedente dello stesso progetto (MedGemma-27B con linee guida cliniche 2026) il RAG serviva a recuperare conoscenza successiva al cutoff del modello; qui ci poniamo la domanda opposta: su benchmark generalisti, che il modello in buona parte già risolve, il RAG aiuta o danneggia? E si può costruire una regola che lo usi solo quando serve, scelta senza guardare il test?

Contributi: (i) un protocollo dev/test rigoroso con test eseguito una sola volta; (ii) un gate selettivo a due livelli (benchmark e singola domanda); (iii) un confronto onesto con il RAG sempre attivo, che risulta dannoso; (iv) codice, indici e risultati grezzi pubblicati (§7).

2. Metodo

2.1 Modello

`mlx-community/Qwen3-14B-8bit` (Apache-2.0), modalità non-thinking, decoding greedy, eseguito con MLX su Apple Silicon. Per ARC la risposta è la lettera con logit massimo tra le opzioni; dai logit delle lettere si ottiene una distribuzione $p(\cdot \mid q)$ sulle k opzioni e la confidenza $c(q) = \max_j p(j \mid q)$.

2.2 Indici e retrieval

Due corpora testuali (CC BY-SA 4.0): `wiki_it` (4588 chunk, Wikipedia italiana, articoli WikiProject Medicine, dump 20231101.it) e `wiki_math` (537 chunk, articoli di matematica di Wikipedia inglese). L'indice BM25 è costruito in memoria; si recuperano i primi 3 brani, troncati a 700 caratteri, preceduti dall'istruzione: "Usali SOLO se sono direttamente utili per rispondere; altrimenti ignorali." Indichiamo con $r(q)$ il punteggio BM25 normalizzato del primo brano (campo `rr` nei file).

2.3 Gate selettivo

Livello benchmark. Per ogni benchmark b e sorgente s, sul dev si calcola il guadagno netto

$$\Delta_b = \sum_{i \in \mathrm{dev}_b} \Big(\mathbb{1}[\hat{y}_i^{\mathrm{RAG}} = y_i] - \mathbb{1}[\hat{y}_i^{\mathrm{base}} = y_i] \Big) .$$

Il RAG è acceso su b solo se $\Delta_b > 0$ e il limite inferiore dell'intervallo bootstrap al 95% è ≥ 0; altrimenti è spento.

Livello domanda (ARC-it). Con sorgente `wiki_math` e soglie $\tau_r = 0,3$, $\tau_c = 0,9$ (e soglia sulla confidenza RAG pari a 0, cioè inattiva):

$$\hat{y}(q) = \begin{cases} \arg\max_j p_{\mathrm{RAG}}(j \mid q) & \text{se } r(q) \geq \tau_r \ \wedge \ c_{\mathrm{base}}(q) < \tau_c \\ \arg\max_j p_{\mathrm{base}}(j \mid q) & \text{altrimenti.} \end{cases}$$

Le soglie sono state scelte per ricerca a griglia sul dev ARC-it.

Insiemi di sviluppo (disgiunti dal test): ARC-it validation (286 domande), GSM8K train (campione di 60, seed 99), IFEval (60 id non presenti tra i 100 del test). **Test:** ARC-it test (1132), GSM8K (150), IFEval (100). Il test è stato eseguito una sola volta (file marcatore `TEST_RUN_ONCE`).

2.4 Statistica

Per confronti appaiati su n item, siano b le domande corrette solo con RAG ("fix") e c quelle corrette solo senza ("break"). Il test di McNemar esatto bilaterale è

$$p = \min\left(1,\ 2\sum_{i=0}^{\min(b,c)} \binom{b+c}{i} 2^{-(b+c)}\right).$$

L'intervallo di confidenza al 95% della differenza di accuratezza $\bar{d} = \frac{1}{n}\sum_i d_i$, con $d_i \in \{-1, 0, 1\}$, è ottenuto con bootstrap percentile ($B = 10\,000$ ricampionamenti, seed 0). La significatività è calcolata offline sui file grezzi.

3. Risultati

3.1 Sviluppo (scelta del gate)

Set dev	**n**	**base**	**RAG**	**fix/break**	**McNemar p**	**bootstrap 95% (pt)**
ARC-it gated (gate tarato qui, ottimistico)	286	0,8916	0,8986	2 / 0	0,50	[0,00; +1,75]
ARC-it sempre attivo, `wiki_it`	286	0,8916	0,8776	4 / 8	0,388	[−3,85; +1,05]
ARC-it sempre attivo, `wiki_math`	286	0,8916	0,8951	7 / 6	1,00	[−2,10; +2,80]
GSM8K gated (`wiki_it`)	60	0,9500	0,9667	1 / 0	1,00	[0,00; +5,00]
IFEval	60	0,90	0,85 (sempre attivo)	–	–	netto 0 → OFF

Decisione finale: ARC-it **ON** (`wiki_math`), GSM8K **OFF**, IFEval **OFF**.

3.2 Test (eseguito una volta)

Set test	**n**	**base**	**RAG**	**fix/break**	**McNemar p**	**bootstrap 95% (pt)**
ARC-it, RAG selettivo	1132	0,9046	**0,9090**	5 / 0	0,0625	[+0,09; +0,88]
ARC-it, RAG sempre attivo `wiki_math`	1132	0,9046	0,8843	19 / 42	0,0044	[−3,36; −0,71]
GSM8K, gate originale (per confronto)	150	0,9333	0,9200	1 / 3	0,625	[−4,00; +1,33]
GSM8K, configurazione finale (OFF)	150	0,9333	0,9333	–	–	–

Set test	n	base	RAG	fix/break	McNemar p	bootstrap 95% (pt)
IFEval (OFF)	100	0,83	0,83	–	–	–

In accuratezza assoluta, ARC-it passa da 1024 a 1029 risposte corrette su 1132 con il RAG selettivo, e scende a 1001 con il RAG sempre attivo. Il gate attiva il RAG su 23 domande su 1132 (2,0%). Abbiamo ricalcolato i valori di ARC-it direttamente dal file `test_raw.json` pubblicato, ottenendo esattamente le cifre in tabella.

4. Discussione

Il RAG indiscriminato danneggia. Su un benchmark in cui il modello è già al 90%, inserire sempre brani Wikipedia rompe 42 risposte giuste a fronte di 19 recuperate; il peggioramento è statisticamente significativo. Questo conferma che il RAG non va considerato un miglioramento automatico.

Il gate selettivo è prudente. Limitando il RAG ai casi incerti con retrieval forte (2% delle domande), il sistema non rompe alcuna risposta e ne corregge 5. Il guadagno (+0,44 pt) è però piccolo: con soli 5 casi discordanti, McNemar non raggiunge 0,05 (p = 0,0625 è il minimo ottenibile con 5/0, cioè $2 \cdot 2^{-5}$). Il CI bootstrap esclude lo zero, ma con così pochi casi discordanti va letto come indicativo, non come prova.

Limiti.

- Effetto piccolo e non significativo a 0,05 su ARC-it; nessun guadagno su GSM8K e IFEval.
- Seed singolo e decoding greedy: nessuna stima della variabilità tra esecuzioni.
- Dev piccoli (60 item per GSM8K e IFEval, 286 per ARC-it): la scelta del gate è fragile.
- Decisione su GSM8K: il run originale (`gate.json`) teneva il RAG attivo su GSM8K (dev +1/60, limite inferiore CI = 0, che formalmente soddisfa la regola "≥ 0"). La configurazione selettiva lo ha spento motivandolo con la dimensione del dev; il file di configurazione cita però anche il risultato di test (−1,33 pt). Per trasparenza segnaliamo che questa scelta è stata formalizzata dopo il run di test, e riportiamo entrambe le righe.
- Path assoluti della macchina originale (Mac) codificati in `qrag.py`: per riprodurre occorre adattarli.
- *`baseline_raw.json`* non pubblicato: le righe no-RAG di GSM8K e IFEval dipendono da un run baseline separato non incluso nel repository dei risultati (in `test_raw.json` il campo `norag` di GSM8K/IFEval è vuoto); per questi due benchmark i numeri base non sono verificabili dai soli file pubblici.
- Corpora generici (Wikipedia medicina IT, matematica EN) non mirati ai benchmark.

5. Riproducibilità

- Codice: https://huggingface.co/odelpa/qwen14-rag-selettivo (`qrag.py`, `gate.json`, `config.json`, `significance.py`, script di costruzione corpora).
- Indici BM25: https://huggingface.co/datasets/odelpa/qwen14-rag-selettivo-index.

- Risultati grezzi (senza testo delle domande): https://huggingface.co/datasets/odelpa/qwen14-rag-v1-risultati (`dev_raw.json`, `test_raw.json`, `significance.jsonl`, `run_q.log`).
- Procedura: `python qrag.py dev` (gate → `gate.json`), `python qrag.py test` (una volta), `python significance.py`.
- Hardware: Mac mini Apple Silicon, MLX; nessun servizio esterno. Run dev 25/10/2026 12:45, test completato 20:46 (ora di Roma).

6. Licenze

Codice Apache-2.0. Modello Qwen3-14B: Apache-2.0. Indici: CC BY-SA 4.0 (testo © autori di Wikipedia). Risultati derivati: CC BY-SA 4.0. Benchmark: ARC italiano `sapienzanlp/arc_italian` (CC BY-SA 4.0), GSM8K (MIT), IFEval (Apache-2.0).

7. Conclusioni

Su Qwen3-14B il RAG BM25 su Wikipedia, applicato sempre, peggiora in modo significativo ARC-it (−2,03 pt). Un gate selettivo scelto solo sul dev evita ogni peggioramento e produce un guadagno piccolo (+0,44 pt, 5 fix / 0 break) non significativo a 0,05; su GSM8K e IFEval il RAG resta spento. Il messaggio pratico è che il RAG va acceso con criterio, misurato su dati separati e riportato con i suoi limiti. Lavori futuri: più seed, dev più ampi, corpora mirati, pubblicazione del baseline grezzo.